

¿Para qué sirve cada archivo de esta carpeta?

Sesión 2 — E-business & E-commerce y Plataformas en Línea

Esta carpeta es el **código base** que el docente expone en clase. No es el trabajo final: es la plantilla que cada alumno debe **adaptar a su propio proyecto** para cumplir los entregables de la sesión ("La Radiografía Técnica" y "El Puente Digital").

Los 3 archivos de código, en una línea:

- `productos-data.js` — los datos del negocio, separados del HTML.
- `puente-digital.js` — la conexión con una API externa.
- `demo-puente-digital.html` — la página que junta ambos y muestra el resultado.

1. `productos-data.js`

MODELADO DE DATOS

En la Sesión 1 los productos de Grafiluz estaban escritos directamente dentro del HTML (`index.html`), mezclados con las etiquetas de presentación. Aquí se extraen a un arreglo de objetos JavaScript: la información del negocio queda **separada** de cómo se ve en pantalla. Así trabaja cualquier sistema real de e-commerce (catálogo, base de datos, API interna).

```
{
  id: "antiestres-01",
  categoria: "Artículos Antiestrés",
  nombre: "Muñeco Antiestrés",
  medidas: "10.5 x 7 cm",
  material: "Espuma de goma",
  colores: ["rojo", "azul", "gris", "blanco"],
  precioReferencialPEN: 12.5,
}
```

Qué debes recrear: reemplazar este arreglo con los productos o servicios de tu propio negocio, manteniendo la misma estructura de campos (o agregando los que tu proyecto necesite).

2. `puente-digital.js`

EL PUENTE DIGITAL

Este es el corazón conceptual de la sesión. Demuestra cómo la tienda se **conecta a un servicio externo mediante una API** para ampliar su alcance, sin que eso sea en sí una venta. El ejemplo consulta el tipo de cambio del día (Soles → Dólares / Euros) usando una API pública gratuita (open.er-api.com, sin necesidad de API key) y convierte los precios del catálogo.

Función	Qué hace
<code>obtenerTasasDeCambio()</code>	Llama a la API externa con <code>fetch</code> y devuelve las tasas de cambio actuales.
<code>convertirCatalogo(productos, tasas, monedas)</code>	Toma el catálogo (de <code>productos-data.js</code>) y calcula los precios convertidos.
<code>renderizarCatalogoConvertido(catalogo, contenedorId)</code>	Pinta el resultado en pantalla, dentro de un contenedor HTML.
<code>iniciarPuenteDigital(productos, contenedorId)</code>	Une todo lo anterior: consulta la API, convierte y muestra. Es la función que se llama al presionar el botón de la demo.

Qué debes recrear: elegir una API externa relevante para *tu* negocio (no tiene que ser tipo de cambio — puede ser clima, redes sociales, geolocalización, cotización de un insumo, o incluso un contador de visitas) y adaptar estas funciones a esa nueva fuente de datos, siguiendo el mismo patrón: `fetch` → `procesar` la respuesta → `mostrar el resultado`.

3. demo-puente-digital.html

VISTA DE LA DEMO

Página que carga `productos-data.js` y `puente-digital.js` y agrega el botón y los contenedores donde se muestra el resultado. Es lo que el docente proyecta en clase, y también el modelo de cómo tu propio proyecto podría mostrar su "Puente Digital" funcionando.

Para probarlo: como el script usa `fetch` para llamar a una API real, el navegador necesita servir el archivo por HTTP (no abrirlo con doble clic como `file://`). Basta con correr un servidor local simple, por ejemplo: `python3 -m http.server` dentro de esta carpeta, y abrir `http://localhost:8000/demo-puente-digital.html`.

Resumen: qué se quiere recrear en esta sesión

1. Tomar tu proyecto de la Sesión 1 y **modelar sus productos/servicios como datos** (tu propio `productos-data.js`).
2. Elegir una API externa que tenga sentido para tu negocio y construir tu propio **"Puente Digital"** siguiendo el patrón de `puente-digital.js`.
3. Armar tu propia página de demo (como `demo-puente-digital.html`) para mostrarlo funcionando.

4. Documentar el resultado en [../Plantilla-Radiografia-Tecnica.md](#) y [../Plantilla-Reporte-Puente-Digital.md](#) .